
B28: SQL SuperHero Program : Session 39

1 message

Vishal Jaiswal <vishaldbdev@gmail.com>
Bcc: satishhattekar1997@gmail.com

29 April 2025 at 00:51

CURSOR

- A cursor is a pointer that points to a result of a query.
- It start with 1st row of data and process row by row.
- it is used for row by row processing

--> whenever we have such a requirement, where we have to go row by row processing, and fetch all records of the dataset, we have to use cursor.

Types of cursor :

- 1. Implicit**
- 2. Explicit**

Implicit cursors

Whenever Oracle executes an SQL statement such as SELECT INTO, INSERT, UPDATE, and DELETE, it automatically creates an implicit cursor. every sql statement which is executing is an implicit cursor.

--> any query which executed by oracle it considered as implicit cursor

Explicit Cursors:

Cursor C1 Is --- **Defining the cursor . syntax --> cursor <cursorname> is and then sql statement**
Select * from employee;

Declare a cursor

Before using an explicit cursor, you must declare it in the declaration section of a block or package as follows:

CURSOR cursor_name IS query;

In this syntax:

First, specify the name of the cursor after the CURSOR keyword.
Second, define a query to fetch data after the IS keyword.

Cursor Attributes

4 types of attributes:

%Found --> Returns TRUE if a successful fetch has been executed. else false
%Not_Found --> opposite of found
%rowCount --> Returns the number of rows fetched.
%ISOPEN --> - Returns TRUE if the cursor is open, FALSE if the cursor is closed.

Cursor should be processed in 3 parts:

open
fetch
close

Open a cursor

Before start fetching rows from the cursor, you must open it. To open a cursor, you use the following syntax:
Open cursor_name;

Fetch from a cursor

The FETCH statement places the contents of the current row into variables. The syntax of FETCH statement is as follows:
Fetch cursor_name into variables;

Closing a cursor

After fetching all rows, you need to close the cursor with the CLOSE statement:
close cursor_name;

```
declare
  cursor c_vishal is
    select * from hr.regions;
```

]- Define here

```
  l_region_id hr.regions.REGION_ID%type;
  l_region_name hr.regions.REGION_NAME%type;
```

Begin

```
  open c_vishal;
```

→ open

Loop

```
  fetch c_vishal into l_region_id, l_region_name;
```

fetch → V-imp

```
  exit when c_vishal%notfound;
```

```
  dbms_output.put_line(l_region_name||' - '||l_region_id);
```

```
end loop;
```

```
close c_vishal;
```

→ close

End;

Statement processed.

Europe - 1

Americas - 2

Asia - 3

Middle East and Africa - 4

Declare

```
  cursor c_customer is
```

```
  select * from co.customers;
```

→ 3 column

```
  l_id co.customers.CUSTOMER_ID%Type;
```

```
  l_email co.customers.EMAIL_ADDRESS%Type;
```

```
  l_name co.customers.FULL_NAME%Type;
```

Begin

```
  open c_customer;
```

loop

```
  fetch c_customer into l_id, l_email, l_name;
```

fetch needs 3 variables

```
  exit when c_customer%notfound;
```

```
  dbms_output.put_line(l_id||' = '||l_email);
```

```
end loop;
```

```
close c_customer;
```

End;

Statement processed.

1 =tammy.bryant@internalmail

2 =roy.white@internalmail

3 =gary.jenkins@internalmail

4 =joshua.white@internalmail

Declare

```
cursor c_customer is  
select * from co.customers;
```

→ 3 column

```
l_id co.customers.CUSTOMER_ID%Type;  
l_email co.customers.EMAIL_ADDRESS%Type;  
l_name co.customers.FULL_NAME%Type;
```

Begin

```
Open c_customer;  
loop  
fetch c_customer into l_id,l_email,l_name;  
exit when c_customer%notfound;  
dbms_output.put_line(l_id||'='||l_email);  
end loop;  
close c_customer;
```

fetch needs 3 variables

End;

if we don't write
this then compiler
does not where
to stop & it
stucks

Statement processed.

```
1 =tammy.bryant@internalmail  
2 =roy.white@internalmail  
3 =gary.jenkins@internalmail
```

Row by Row Processing.

```
1 Declare  
2 cursor c_vishal is  
3 select * from hr.regions;  
4  
5 l_region_id hr.regions.REGION_ID%type;  
6 l_region_name hr.regions.REGION_NAME%Type;  
7  
8 Begin  
9 Open c_vishal;  
10 Loop  
11 Fetch c_vishal Into l_region_id,l_region_name;  
12 exit when c_vishal%notfound;  
13 dbms_output.put_line(l_region_name||' - '||l_region_id);  
14 End Loop;  
15 Close c_vishal;  
16 End;  
17  
18 Statement processed.  
19 Europe - 1  
20 Americas - 2  
21 Asia - 3  
Middle East and Africa - 4
```

Explicit cursor

} - define cursor

2 columns = 2 variables

→ This query executes in memory

↳ Printing the first row
& so for 1 by 1
each row

Declare

```
cursor c_ankur is
```

```
select department_id, max(salary) from emp group by department_id;
```

```
l_id number;
```

```
l_sal number;
```

Begin

```
open c_ankur;
```

```
loop
```

```
fetch c_ankur into l_id, l_sal;
```

```
exit when c_ankur%notfound;
```

```
update emp set salary = l_sal where department_id = l_id;
```

```
End Loop;
```

```
close c_ankur;
```

End;

Statement processed.

]- initialize

→ open
Fetch (one row at a time)

→ update that row

→ Row by Row processing

See this simple cursor Loop explanation:

declare

```
cursor c1 is
```

```
select * from hr.employees;
```

```
r c1%rowtype;
```

Begin

```
open c1;
```

```
Loop
```

```
fetch c1 into r;
```

```
exit when c1%notfound;
```

```
dbms_output.put_line(r.first_name);
```

```
end loop;
```

```
close c1;
```

End;

declare

```
cursor c1 is
```

```
select * from hr.employees;
```

```
r c1%rowtype;
```

Begin

```
open c1;
```

```
Loop
```

```
fetch c1 into r;
```

```
exit when c1%notfound;
```

```
dbms_output.put_line(r.first_name);
```

```
end loop;
```

```
close c1;
```

End;

Statement processed.

Steven

Neena

Lex

Alexander

]- cursor declare

→ cursor object

→ Fetch cursor

→ close cursor

]- output

Open
Cursor

Exit when
done

```

Declare
    Cursor c1 Is
        select * from employees;

    r c1%rowtype;

Begin
    Open C1;
    Loop
        Fetch C1 into r;
        Exit when C1%Notfound;
        update employees set salary = salary+ 1000 where employee_id = r.EMPLOYEE_ID;
        dbms_output.put_line('salary updated for '||r.EMPLOYEE_ID);
    End Loop;
End;

```

For every row it process & update table

```

Statement processed.
salary updated for 100
salary updated for 101
salary updated for 102
salary updated for 103

```

Difference Between %TYPE and %ROWTYPE

%TYPE Vs. %RowType

- %TYPE provides the data type of a variable or a database column to that variable.
- %ROWTYPE provides the record type that represents a entire row of a table or view or columns selected in the cursor.

The Advantages are:

- I Need not to know about variable's data type.
- If the database definition of a column in a table changes, the data type of a variable changes accordingly.

Write a pl/sql block to print the number of employees in each department.

--> Multiple records, need to be printed, means cursor. because normal select can print only 1.

```

Declare
    cursor c1 is
        select department_id, count(*) from hr.employees group by department_id;
    l_id number;
    l_count number;

Begin
    open C1;
    Loop
        Fetch c1 into l_id, l_count;
        exit when c1%notfound;
        dbms_output.put_line('department '||l_id||' has total '||l_count|| 'employees');
    end loop;

close c1;
End;

```

```

Declare
    cursor c1 is
        select department_id, count(*) from hr.employees group by department_id;
        l_id number;
        l_count number;

Begin
    open c1;
    Loop
        Fetch c1 into l_id, l_count;
        exit when c1%notfound;
        dbms_output.put_line('department ' || l_id || ' has total ' || l_count || ' employees');
    end loop;
    close c1;
End;

```

Handwritten notes:

- open* (next to open c1;)
- Fetch* (next to Fetch c1 into l_id, l_count;)
- close* (next to close c1;)
- dummy is here in cursor* (pointing to the cursor declaration)
- most imp line* (pointing to the fetch statement)
- Printed* (pointing to the dbms_output.put_line statement)

Facts About Cursor:

- Cursor Is a memory Object that will be stored inside RAM memory.
- Main advantage of using a cursor is when we need row by row processing we need cursor .
- The TOO_MANY_ROWS error occurs when a select gives more than requested data, Cursor help us to avoid this error.

Implicit

- Oracle Internally creates Implicit cursor
- Can not give a name to implicit cursor
- Whenever any statement executes (select/insert/update/delete) it creates Implicit cursor
- status of Implicit cursor can be found via sql%rowcount

Explicit

- Declare Cursor to Initializing memory
- Open cursor for Allocation Memory
- Fetch cursor for retrieving records
- Close cursor for releasing memory

Handwritten note: used with update (IMPLICIT CURSOR)

```

1  begin
2      update emp set salary = 100 where department_id = 60;
3      dbms_output.put_line('total records updated ' || SQL%ROWCOUNT);
4  End;
5
6

```

Handwritten note: cursor attribute Row count (pointing to SQL%ROWCOUNT)

Statement processed.
total records updated 5

See the above example, SQL%ROWCOUNT is an attribute of cursor. Used with a simple update statement , means IMPLICIT. Every sql executed is a CURSOR.

[Normal v. type]

```
11 DECLARE
12 l_custid co.customers.CUSTOMER_ID%type;
13 l_email co.customers.EMAIL_ADDRESS%type;
14 l_name co.customers.FULL_NAME%type;
15 BEGIN
16 SELECT * into l_custid, l_email, l_name
17 FROM co.customers
18 WHERE CUSTOMER_ID = 1;
19 dbms_output.put_line('Customer ID: ' || l_custid);
20 dbms_output.put_line('Customer Email: ' || l_email);
21 dbms_output.put_line('Customer Name: ' || l_name);
22 END;
```

Statement processed.
Customer ID: 1
Customer Email: tammy.bryant@internalmail
Customer Name: Tammy Bryant

3 column.
So need
3 variables

All 3
here
↓
Need to
declare
them

```
1 DECLARE
2 customer_rec co.customers%rowtype;
3 BEGIN
4 SELECT * into customer_rec
5 FROM co.customers
6 WHERE CUSTOMER_ID = 1;
7 dbms_output.put_line('Customer ID: ' || customer_rec.CUSTOMER_ID);
8 dbms_output.put_line('Customer Email: ' || customer_rec.EMAIL_ADDRESS);
9 dbms_output.put_line('Customer Name: ' || customer_rec.FULL_NAME);
10 END;
```

Statement processed.
Customer ID: 1
Customer Email: tammy.bryant@internalmail
Customer Name: Tammy Bryant

All table columns declared
here in
rowtype

All 3 come here into rowtype

Use here
variable - column name

Parameterized Cursor:

Declare
cursor cur_emp(p_dept_id number) is
Select * from hr.employees
where department_id = p_dept_id;

r_emp hr.employees%rowtype;

Begin
open cur_emp(90);
Loop
Fetch cur_emp into r_emp;
exit when cur_emp%notfound;

dbms_output.put_line(r_emp.first_name||' : '||r_emp.salary);
End Loop;
close cur_emp;

End;

```
Declare
  cursor cur_emp(p_dept_id number) is
  Select * from hr.employees
  where department_id = p_dept_id;

  r_emp hr.employees%rowtype;

Begin
  open cur_emp(90);
  Loop
  Fetch cur_emp into r_emp;
  exit when cur_emp%notfound;

  dbms_output.put_line(r_emp.first_name||' : '||r_emp.salary);
  End Loop;
  close cur_emp;

End;
```

Handwritten notes:
I/P Parameter (pointing to p_dept_id)
Value passed (pointing to 90)
Cursor run for dept 90 only (pointing to the output)

```
Statement processed.
Steven : 24000
Neena : 17000
Lex : 17000
```

In Below example, same cursor run in same program for 2 different parameter.

```
Declare
  cursor cur_emp(p_dept_id number) is
  Select * from hr.employees
  where department_id = p_dept_id;

  r_emp hr.employees%rowtype;

Begin
  open cur_emp(90);
  Loop
  Fetch cur_emp into r_emp;
  exit when cur_emp%notfound;

  dbms_output.put_line(r_emp.first_name||' : '||r_emp.salary);
  End Loop;
  close cur_emp;
  dbms_output.put_line('extra records');
  open cur_emp(100);
  Loop
  Fetch cur_emp into r_emp;
  exit when cur_emp%notfound;

  dbms_output.put_line(r_emp.first_name||' : '||r_emp.salary);
  End Loop;
  close cur_emp;

End;
```

Cursor FOR UPDATE/ WHERE CURRENT OF EXP > 5+

Sometimes, you want to lock a set of rows before you can update them in your program. Oracle provides the FOR UPDATE clause of the SELECT statement in an updatable cursor to perform this kind of locking mechanism. Syntax as below:

```
CURSOR cursor_name IS
  SELECT select_clause
  FROM from_clause
  WHERE where_clause
  FOR UPDATE;
```

Once you open the cursor, Oracle will lock all rows selected by the SELECT ... FOR UPDATE statement in the tables specified in the FROM clause. And these rows will remain locked until the cursor is closed or the transaction is completed with either COMMIT or ROLLBACK.


```
create table employ as select * from hr.employees
```

```
declare
  cursor c1 is
    select * from employ where department_id = 90 for update;

  r c1%rowtype;
Begin
  open c1;
  loop
    fetch c1 into r;
    exit when c1%notfound;
    update employ set salary = 500 where current of c1;
    dbms_output.put_line(r.first_name);
  End Loop;
  close c1;

End;
```

```
declare
  cursor c1 is
    select * from employ where department_id = 90 for update;

  r c1%rowtype;
Begin
  open c1;
  loop
    fetch c1 into r;
    exit when c1%notfound;
    update employ set salary = 500 where current of c1;
    dbms_output.put_line(r.first_name);
  End Loop;
  close c1;

End;
```

Look for these records

used here

```
Statement processed.
Steven
Neena
Lex
```

If you plan on updating or deleting records that have been referenced by a SELECT FOR UPDATE statement, you can use the WHERE CURRENT OF statement.

The syntax for the WHERE CURRENT OF statement in Oracle/PLSQL is either:

```
UPDATE table_name
SET set_clause
WHERE CURRENT OF cursor_name;
OR
```

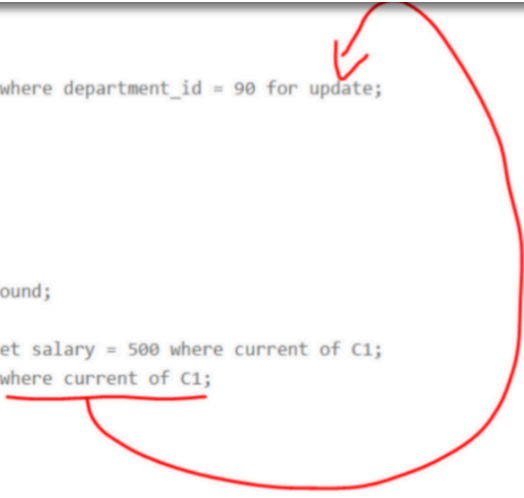
```
DELETE FROM table_name
WHERE CURRENT OF cursor_name;
```

```
Declare
  cursor c1 is
    select * from employ where department_id = 90 for update;

  r c1%rowtype;
Begin
  Open C1;
  Loop
    Fetch c1 Into r;
    exit when C1%notfound;

    -- update employ set salary = 500 where current of C1;
    delete from employ where current of C1;

  End Loop;
End;
```



Statement processed.